

Hybrid Search Text and Vector Guide

RAG Search Service implementation reference

Purpose and confirmed POC status

The RAG Search POC implements both lexical text retrieval and semantic vector retrieval. In the default hybrid mode, the service runs both retrieval paths, oversamples their candidate sets, merges candidates by `chunk_id`, applies reciprocal-rank fusion by default, filters low-quality results, reranks the survivors, and returns the final `top_k` chunks.

The POC contains three distinct matching mechanisms:

Mechanism	Used in the POC	Role
Indexed PostgreSQL full-text search with <code>tsvector</code>	Yes	Primary lexical candidate generator
Direct raw <code>chunk_text</code> substring matching with <code>LIKE</code>	Yes	Supplemental exact-phrase selection and score bonus
Semantic embedding search with <code>pgvector</code>	Yes	Primary semantic candidate generator

The direct `LIKE` logic does not replace PostgreSQL full-text search. It supplements the indexed lexical path. Hybrid search combines the indexed lexical candidates with semantic candidates.

Section 1 Text Search

Text search paths used in the POC

Text-search path	SQL form	POC usage	Result
Indexed lexical search	<code>c.search_vector @@ sq.tsq</code>	Yes, primary text retrieval	Selects chunks whose normalized lexemes match the normalized NLQ lexemes
Full-text ranking	<code>ts_rank_cd(c.search_vector, sq.tsq)</code>	Yes	Produces the base keyword relevance score
Direct complete-query bonus	<code>lower(c.chunk_text) LIKE '%' lower(:query) '%'</code>	Yes	Adds 1.0 when the complete normalized query is a literal substring; does not select an ordinary-query row by itself
Direct exact-quote selection	<code>query_intent = 'exact_quote'</code> <code>AND lower(c.chunk_text) LIKE ...</code>	Yes	Can select a chunk directly for exact-quote intent
Resource-field matching	<code>LIKE</code> against title, author, category, and tags	Yes	Finds resource metadata matches and applies configured score boosts

Natural language question handling

The application preprocesses the NLQ before calling PostgreSQL. `QueryPreprocessor.understand()` in `app/search/query_preprocessor.py:36-60` collapses whitespace, extracts quoted text, removes a limited set

of conversational prefixes, removes prefixes such as `book:` or `document:`, normalizes configured aliases, creates meaningful tokens, and classifies the query intent.

NLQ stage	Example	Implementation
Original input	Can you tell me how long products are covered under warranty?	API request query
Application normalization	Removes a recognized conversational prefix and normalizes configured aliases	<code>app/search/query_preprocessor.py:36-74</code>
Ordinary PostgreSQL query	<code>websearch_to_tsquery('english', :query)</code>	<code>app/repositories/rag_search_repository.py:104-108</code>
Quoted PostgreSQL query	<code>phraseto_tsquery('english', :query)</code>	Same repository lines
Conceptual lexemes	<code>long & product & cover & warranti</code>	Produced by PostgreSQL English dictionaries; exact output depends on configuration

The complete NLQ does not need to appear verbatim in the chunk. PostgreSQL tokenizes the query, removes configured stop words, and stems words. Lexical retrieval still requires compatible lexemes. A paraphrase with entirely different vocabulary may not match; semantic search covers that case.

The current application cleanup is conservative and limited to configured patterns. Its `normalized_terms` support intent detection and reranking, but the SQL query receives the normalized query string. A filler expression not recognized by application preprocessing is removed only if PostgreSQL treats it as a stop word. This is a known improvement area: broaden tested prefix cleanup or construct the lexical query from conservatively selected content terms, then validate the change with a labeled relevance set.

The `search\_vector` column

`rag_document_chunks.search_vector` is a PostgreSQL `tsvector`. It is not an AI embedding. It contains normalized lexemes and their positions.

Code or definition	Meaning	POC status and source
<code>search_vector tsvector GENERATED ALWAYS AS (to_tsvector('english', coalesce(chunk_text, ''))) STORED</code>	PostgreSQL derives a lexical representation from <code>chunk_text</code> and stores it on the same row	Used; <code>modules/rag-ingest-service/sql/schema.sql:309</code>
<code>Computed("to_tsvector('english', coalesce(chunk_text, ''))", persisted=True)</code>	SQLAlchemy declares the column as database-computed	Used; <code>modules/rag-ingest-service/app/db/models.py:207-210</code>
<code>CREATE INDEX ... USING gin (search_vector)</code>	Creates the inverted index used for scalable full-text lookup	Used; <code>modules/rag-ingest-service/sql/schema.sql:452</code>

Illustrative conversion:

Input or output	Value
<code>chunk_text</code>	The products are covered under warranties
<code>to_tsvector('english', chunk_text)</code>	<code>'cover':4 'product':2 'warranti':6</code>

The position numbers identify token locations; they are not semantic coordinates.

When the text search vector is created

Step	Code or database action	Outcome
Chunk insertion	Worker supplies chunk_text at modules/rag-ingest-service/app/workers/rag_ingestion_worker.py:180-186	A new chunk row is written
Generated-column evaluation	PostgreSQL runs to_tsvector('english', chunk_text)	search_vector is computed automatically
Stored value	Column is declared STORED	The computed representation is persisted
Index maintenance	PostgreSQL updates the GIN index	Lexemes point to the new chunk row
Later text update	PostgreSQL reevaluates the generated expression	search_vector and related GIN entries stay synchronized automatically

Application code does not manually set search_vector. The database creates it when chunk_text is inserted and recalculates it when chunk_text changes.

Direct and indirect use of `chunk\_text`

Usage	Directly reads chunk_text	Selects candidates	Changes score	Details
search_vector @@ tsquery	No, indirect	Yes	Via ts_rank_cd	Primary indexed lexical path; chunk_text created the vector earlier
Complete-query substring bonus	Yes	No, not alone for ordinary queries	Yes, +1.0	app/repositories/rag_search_repository.py:140
Exact-quote substring condition	Yes	Yes, for exact-quote intent	Yes	app/repositories/rag_search_repository.py:151
Quality filtering and local reranking	Yes	Post-retrieval	Yes	Application evaluates readable text
Snippet and response construction	Yes	No	No	Returns readable evidence to the caller

The main indirect path is:

Code snippet
chunk_text -> generated search_vector -> GIN lookup -> matching chunk row

The supplemental direct path is:

Code snippet
<pre>-- Literal complete-query bonus + CASE WHEN lower(c.chunk_text) LIKE '%' lower(:query) '%' THEN 1.0 ELSE 0.0 END</pre>

Code snippet

```
-- Exact-quote candidate condition
OR (
  :query_intent = 'exact_quote'
  AND lower(c.chunk_text) LIKE '%' || lower(:query) || '%'
)
```

Keyword SQL used in the POC

SQL fragment	Purpose	POC source
websearch_to_tsquery('english', :query)	Converts ordinary NLQ text into a forgiving web-style tsquery	app/repositories/rag_search_repository.py:107
phraseto_tsquery('english', :query)	Preserves phrase adjacency for exact-quote intent	app/repositories/rag_search_repository.py:106
c.search_vector @@ sq.tsq	Selects lexically matching chunk rows	app/repositories/rag_search_repository.py:150
ts_rank_cd(c.search_vector, sq.tsq)	Computes cover-density lexical relevance	app/repositories/rag_search_repository.py:134
lower(c.chunk_text) LIKE ...	Checks a literal raw-text substring	app/repositories/rag_search_repository.py:140,151

Resource-field score boosts add 10 for an exact title, 5 for a title substring, 3 for an author substring, 2 for a category substring, 2 for a tag substring, and 1 for a literal chunk substring. These values are defined at `app/repositories/rag_search_repository.py:134-140`.

Plain text search performance

Technique	Typical performance	Index behavior	POC recommendation
LIKE 'prefix%'	Often reasonable for selective prefix searches	A suitable B-tree/operator-class configuration may help	Appropriate for controlled prefix fields, not general NLQ retrieval
LIKE '%substring%'	Can scan many rows without a specialized index	Ordinary B-tree cannot efficiently support the leading wildcard	Keep supplemental and selective
LIKE '%substring%' with pg_trgm GIN/GiST	Better for substantial strings with extractable trigrams	PostgreSQL can use trigram index entries	POC has pg_trgm and a GIN trigram index on chunk_text
tsvector @@ tsquery with GIN	Designed for repeated document search	Inverted index maps lexemes to row identifiers	Use as the primary lexical path, as the POC does

The schema enables `pg_trgm` at `modules/rag-ingest-service/sql/schema.sql:6`, creates a trigram GIN index on `chunk_text` at `:451`, and creates the full-text GIN index at `:452`. PostgreSQL decides which index to use from query shape, selectivity, table statistics, and estimated cost. Confirm real plans with `EXPLAIN (ANALYZE, BUFFERS)`.

Performance cautions:

- A leading-wildcard LIKE predicate can be expensive when no trigram index is usable.
- Very short patterns contain few trigrams and may have poor selectivity.

- Large result sets still cost time even when an index finds them quickly.
- tsvector search avoids reparsing every stored document during each request because document-side normalization occurs when the row is written.
- The fixed 'english' configuration is inappropriate for non-English content; multilingual indexing requires a compatible per-language design.
- Ranking, joins, filters, and sorting should be measured together rather than benchmarking the text predicate alone.

Representative verification SQL:

Code snippet

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.chunk_id, ts_rank_cd(c.search_vector, q.tsq)
FROM public.rag_document_chunks c
CROSS JOIN (
    SELECT websearch_to_tsquery('english', :query) AS tsq
) q
WHERE c.search_vector @@ q.tsq
ORDER BY ts_rank_cd(c.search_vector, q.tsq) DESC
LIMIT 50;
```

Section 2 Semantic and Vector Search

Embedding vector and POC usage

`rag_chunk_embeddings.vector` is a dense pgvector value created by an embedding model. The current schema declares `vector(768)`. Unlike `tsvector`, its individual values are learned numeric coordinates and are not directly readable.

Property	Text search_vector	Semantic embedding vector
Type	PostgreSQL tsvector	pgvector vector(768)
Meaning	Lexemes and token positions	Dense learned semantic representation
Creator	PostgreSQL generated column	Ollama or OpenAI embedding provider
Input	Raw chunk_text	Enriched resource and chunk text
Recalculated automatically after text change	Yes	No; re-indexing must regenerate it
Match	@@ tsquery	<=> query_vector cosine distance
Index	GIN	HNSW with vector_cosine_ops
Used in this POC	Yes	Yes

Natural language question handling

Stage	Action	POC source
Preprocess	Normalize whitespace, recognized prefixes, quotation handling, and configured aliases	app/search/query_preprocessor.py: 36 - 74

Stage	Action	POC source
Create query embedding	Send the normalized query to the configured embedding provider	app/services/search_service.py:134-148
Enforce compatibility	Filter stored rows by provider, model, version, and dimension	app/repositories/rag_search_repository.py:80-85
Compare vectors	Order stored embeddings by cosine distance	app/repositories/rag_search_repository.py:75,87-88
Recover readable evidence	Join embedding chunk_id to rag_document_chunks.chunk_id	app/repositories/rag_search_repository.py:76-78

The semantic path embeds the normalized NLQ as a whole. It does not require exact token overlap. Conversational cleanup still reduces noise, but the model is generally more tolerant of natural phrasing than lexical search.

When document embeddings are created

Step	Implementation	Outcome
Identify missing embeddings	Worker checks existing chunk/provider/model/version combinations	modules/rag-ingest-service/app/workers/rag_ingestion_worker.py:297-336
Build embedding input	Adds title, author, category, tags, description, section, heading path, page, and cleaned chunk text when present	modules/rag-ingest-service/app/services/embedding_input_service.py:11-57
Generate vectors	Configured provider embeds batches	modules/rag-ingest-service/app/workers/rag_ingestion_worker.py:347-367
Validate	Check returned count and configured dimension	modules/rag-ingest-service/app/services/embedding_service.py:137-145
Store and link	Persist vector and model identity with the chunk's chunk_id	modules/rag-ingest-service/app/workers/rag_ingestion_worker.py:383-395

Vector SQL used in the POC

SQL fragment	Purpose	POC source
1 - (e.vector <=> CAST(:query_vector AS vector)) AS vector_score	Converts cosine distance to a similarity-style score	app/repositories/rag_search_repository.py:75
JOIN ... c ON c.chunk_id = e.chunk_id	Resolves a matched vector to its readable chunk	app/repositories/rag_search_repository.py:77
e.embedding_provider = :provider	Prevents cross-provider comparison	app/repositories/rag_search_repository.py:82
Model, version, and dimension predicates	Prevent incompatible embedding spaces	app/repositories/rag_search_repository.py:83-85

SQL fragment	Purpose	POC source
ORDER BY e.vector <=> CAST(:query_vector AS vector)	Returns nearest stored embeddings first	app/repositories/rag_search_repository.py:87
LIMIT :limit	Bounds the candidate set	app/repositories/rag_search_repository.py:88

The vector is never decoded into text. The matching embedding row carries chunk_id; SQL joins that identifier to the original chunk row and selects chunk_text, location information, and metadata.

Vector performance notes

The schema creates an HNSW cosine index at modules/rag-ingest-service/sql/schema.sql:477. HNSW provides approximate nearest-neighbor retrieval and normally gives a favorable latency-recall tradeoff for growing collections. Exact vector scans provide maximal recall but become more expensive as embedding count and dimension grow.

Filtered approximate search can return fewer useful candidates because filtering and nearest-neighbor traversal interact. The POC oversamples in hybrid mode, but application oversampling cannot recover a vector row that PostgreSQL did not expose from the approximate scan. For heavily filtered workloads, measure recall and consider pgvector iterative scans, partial indexes, partitioning, or tuned HNSW search breadth.

The schema fixes the physical vector column at 768 dimensions. Selecting a model with a different output dimension requires a database and index migration; changing only application configuration is insufficient.

Hybrid combination used in the POC

Hybrid stage	POC behavior	Source
Candidate generation	Runs keyword and vector retrieval	app/services/search_service.py:118-200
Oversampling	Default factor is 5, bounded by maximum top_k	app/services/search_service.py:130-132; app/core/config.py:36
Fusion	Reciprocal-rank fusion by default; weighted normalized fusion is available	app/services/hybrid_search_service.py:5-89
Candidate identity	Merges results by chunk_id	app/services/hybrid_search_service.py
Quality controls	Removes empty, boilerplate, corrupted, and explicitly non-searchable chunks	app/search/result_quality.py
Reranking	Applies local deterministic relevance heuristics	app/services/reranking_service.py
Final selection	Applies minimum score, stable ordering, and final top_k	app/services/ranking_service.py

Default fusion settings are 0.70 vector weight, 0.30 keyword weight, RRF strategy, and RRF K of 60 (app/core/config.py:34-38). With RRF, ranks are more important than incompatible raw score scales.

Final confirmed architecture

Question	Confirmed answer
----------	------------------

Question	Confirmed answer
Does the POC use indexed text search?	Yes. PostgreSQL tsvector, tsquery, @@, ts_rank_cd, and a GIN index are used.
Does the POC directly query raw chunk_text?	Yes. Direct LIKE logic supports exact-quote selection and a small literal-query score bonus.
Does the POC use semantic vector search?	Yes. It embeds the NLQ and performs pgvector cosine-distance search over compatible chunk embeddings.
Does hybrid mode run both primary retrieval paths?	Yes. It runs text and vector retrieval, oversamples, merges by chunk_id, and fuses the rankings.
Does the vector convert back into text?	No. Its chunk_id joins to the original readable chunk row.
Is direct LIKE '%NLQ%' the primary text-search design?	No. It is supplemental; indexed tsvector matching is primary.

Primary references

- PostgreSQL text search controls: <https://www.postgresql.org/docs/current/textsearch-controls.html>
- PostgreSQL text search tables and indexes: <https://www.postgresql.org/docs/current/textsearch-tables.html>
- PostgreSQL preferred full-text index types: <https://www.postgresql.org/docs/current/textsearch-indexes.html>
- PostgreSQL trigram matching: <https://www.postgresql.org/docs/current/pgtrgm.html>
- pgvector: <https://github.com/pgvector/pgvector>